
DAY 4

Kubernetes PostgreSQL 18 Backup & Disaster Recovery

Step-by-Step Lab Manual
Live Cluster First → Disposable backup-lab Second

Namespace	postgres-ha
Cluster	acid-postgres-ha (Zalando Operator + Patroni)
PostgreSQL	18
Tooling	pgBackRest, pg_dump, kubectl
Duration	3–4 hours
Date	2026-09-27

■ SAFETY WARNING

This lab works on a real Patroni cluster. Never run restore commands against the original PVCs of acid-postgres-ha-0/1/2. Always identify the current primary first. All physical restore and PITR practice must be performed on a new test cluster or the disposable backup-lab.

Table of Contents

- Part 1 – Live Cluster (acid-postgres-ha)
 - 1.1 Inspect the Live Cluster
 - 1.2 Logical Backup from Live Cluster
 - 1.3 Check WAL Archiving Status
 - 1.4 Trigger Operator Native Backup
 - 1.5 Create Test Cluster for Safe Restore
- Part 2 – Disposable Lab (backup-lab) – Full pgBackRest
 - 2.1 Create Lab Resources
 - 2.2 Configure Stanza & WAL Archiving
 - 2.3 Logical Backup & Restore
 - 2.4 Full, Incremental & Differential Backups
 - 2.5 Full Restore
 - 2.6 Point-in-Time Recovery (PITR)
 - 2.7 Compression, Retention & Cleanup
- Coverage Checklist
- References

PART 1 – Live Cluster (acid-postgres-ha)

All commands in this part target the real Patroni-managed cluster. Focus is on safe inspection, logical backups, and restore practice into a separate test cluster.

1.1 Inspect the Live Cluster

```
kubectl config set-context --current --namespace=postgres-ha

kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc

# Identify the current Primary (do not assume pod-0)
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "=== $p ==="
  kubectl exec $p -- curl -s http://127.0.0.1:8008/ | jq -r '.role // .state'
done
```

Set the primary variable after checking:

```
PRIMARY=acid-postgres-ha-0 # ← change after checking
```

1.2 Logical Backup from Live Cluster

Full cluster logical backup:

```
kubectl exec $PRIMARY -- pg_dumpall -U postgres \
--clean --if-exists \
> postgres-ha-full-$(date +%Y%m%d-%H%M).sql
```

Single database logical backup (recommended):

```
kubectl exec $PRIMARY -- pg_dump -U postgres -Fc -d postgres \
-f /tmp/postgres.dump

kubectl cp postgres-ha/$PRIMARY:/tmp/postgres.dump \
./postgres-$(date +%Y%m%d).dump
```

1.3 Check WAL Archiving Status

```
kubectl exec $PRIMARY -- psql -U postgres -c "SHOW archive_mode;"
kubectl exec $PRIMARY -- psql -U postgres -c "SHOW archive_command;"
kubectl exec $PRIMARY -- psql -U postgres -c "SELECT * FROM pg_stat_archiver;"

kubectl get postgresql acid-postgres-ha -o yaml | grep -A 40 -E 'backup|wal|archive|clone'
```

1.4 Trigger Operator Native Backup (if configured)

```
kubectl annotate postgresql acid-postgres-ha \
force-backup.acid.zalan.do=$(date +%s) --overwrite
```

■ NOTE

Monitor progress with: `kubectl logs -l application=spilo -c postgres --tail=100 | grep -iE 'backup|wal-g|archive'`

1.5 Create Test Cluster for Safe Restore

Save the following as **acid-recovery-test.yaml**:

```
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-recovery-test
  namespace: postgres-ha
spec:
  teamId: "acid"
  volume:
    size: 10Gi
  numberOfInstances: 1
  users:
    zalando: []
  databases:
    recoverydb: zalando
  postgresql:
    version: "18"
```

```
kubectl apply -f acid-recovery-test.yaml
kubectl wait --for=condition=Ready pod -l cluster-name=acid-recovery-test --timeout=10m
```

```
RECOVERY_POD=$(kubectl get pod -l cluster-name=acid-recovery-test \
  -o jsonpath='{.items[0].metadata.name}')
echo "Recovery pod: $RECOVERY_POD"
```

Logical Restore into the test cluster:

```
kubectl cp ./postgres-*.dump postgres-ha/$RECOVERY_POD:/tmp/postgres.dump

kubectl exec $RECOVERY_POD -- \
  pg_restore -U postgres -d recoverydb -v --clean --if-exists /tmp/postgres.dump
```

PART 2 – Disposable Lab (backup-lab)

This part creates an independent PostgreSQL 18 + pgBackRest environment. All advanced enterprise features (full/incr/diff, repository, compression, retention, PITR) are practiced here safely without touching the live Patroni cluster.

2.1 Create Lab Resources

Check storage first:

```
kubectl get storageclass
```

Save the following as **backup-lab.yaml** (Part A – PVC + ConfigMap):

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: backup-lab-repo
  namespace: postgres-ha
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 3Gi
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: backup-lab-config
  namespace: postgres-ha
data:
  pgbackrest.conf: |
    [lab]
    pg1-path=/var/lib/postgresql/data/pgdata
    pg1-port=5432

    [global]
    repo1-path=/repo
    repo1-retention-full=2
    compress-type=gz
    compress-level=3
    log-level-console=info
```

Continue the same file (Part B – StatefulSet):

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: backup-lab
  namespace: postgres-ha
spec:
  serviceName: backup-lab
  replicas: 1
  selector:
    matchLabels:
      app: backup-lab
  template:
    metadata:
      labels:
        app: backup-lab
    spec:
      initContainers:
        - name: permissions
          image: postgres:18-trixie
          command: [sh, -ec]
          args: ["chown -R 999:999 /repo /var/lib/postgresql/data"]
          securityContext:
            runAsUser: 0
          volumeMounts:
            - name: repo
              mountPath: /repo
            - name: data
              mountPath: /var/lib/postgresql/data
      containers:
        - name: postgres
          image: postgres:18-trixie
          command: [bash, -ec]
          args:
            - |
              apt-get update && apt-get install -y pgbackrest
              pgbackrest version
              exec docker-entrypoint.sh postgres \
                -c archive_mode=on \
                -c "archive_command=pgbackrest --stanza=lab archive-push %p"
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: backup-lab-auth
                  key: password
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
            - name: repo
              mountPath: /repo
            - name: config
              mountPath: /etc/pgbackrest
      volumes:
        - name: repo
          persistentVolumeClaim:
            claimName: backup-lab-repo
        - name: config
          configMap:
            name: backup-lab-config
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 2Gi

```

```
kubectl -n postgres-ha create secret generic backup-lab-auth \
  --from-literal=password="$(openssl rand -hex 20)"

kubectl apply -f backup-lab.yaml
kubectl -n postgres-ha rollout status statefulset/backup-lab --timeout=10m
kubectl -n postgres-ha logs backup-lab-0 -c postgres --tail=30
```

■ **NOTE**

If apt reports that pgBackRest does not support PostgreSQL 18, stop and use an image that contains a compatible version. If the pod is Pending, check StorageClass and PVC capacity.

2.2 Configure Stanza & WAL Archiving

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab stanza-create

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c \
  "SELECT archived_count, failed_count FROM pg_stat_archiver;"
```

Expected: stanza-create and check succeed, archive_mode=on, archived_count increases.

2.3 Logical Backup & Restore (Lab)

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "CREATE DATABASE drlab;"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "CREATE TABLE orders(id int PRIMARY KEY, note text);
  INSERT INTO orders VALUES (1,'full');"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pg_dump -U postgres -Fc -d drlab -f /tmp/drlab.dump

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "CREATE DATABASE drlab_copy;"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pg_restore -U postgres -d drlab_copy /tmp/drlab.dump

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab_copy -c "TABLE orders;"
```

Expected: drlab_copy contains row 1.

2.4 Full, Incremental & Differential Backups

```
# Full Backup (label ends with F)
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup

# Incremental (label ends with I)
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (2,'incremental');"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup

# Differential (label ends with D)
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (3,'differential');"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup

# Verify
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
```

Expected: info shows F, I and D backups with correct dependency relationships.

2.5 Full Restore

pgBackRest restore requires PostgreSQL to be stopped. We scale the StatefulSet to zero, mount the PVCs in a temporary recovery pod, restore, then scale back up.

```
# Force archive and stop the lab pod
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (4,'archived WAL');"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"

kubectl -n postgres-ha scale statefulset/backup-lab --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 --timeout=180s
```

Save as **backup-lab-recovery.yaml**:

```
apiVersion: v1
kind: Pod
metadata:
  name: backup-lab-recovery
  namespace: postgres-ha
spec:
  restartPolicy: Never
  containers:
  - name: toolbox
    image: postgres:18-trixie
    command: [bash, -ec]
    args: ["apt-get update && apt-get install -y pgbackrest && sleep infinity"]
    volumeMounts:
    - name: data
      mountPath: /var/lib/postgresql/data
    - name: repo
      mountPath: /repo
    - name: config
      mountPath: /etc/pgbackrest
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data-backup-lab-0
  - name: repo
    persistentVolumeClaim:
      claimName: backup-lab-repo
  - name: config
    configMap:
      name: backup-lab-config

kubectl apply -f backup-lab-recovery.yaml
kubectl -n postgres-ha wait --for=condition=Ready pod/backup-lab-recovery --timeout=10m

kubectl -n postgres-ha exec backup-lab-recovery -- \
  bash -ec "mv /var/lib/postgresql/data/pgdata \
  /var/lib/postgresql/data/pgdata.before-restore && \
  install -d -o postgres -g postgres /var/lib/postgresql/data/pgdata"

kubectl -n postgres-ha exec backup-lab-recovery -- \
  gosu postgres pgbackrest --stanza=lab \
  --type=default --target-action=promote restore

kubectl -n postgres-ha delete pod backup-lab-recovery --wait=true
kubectl -n postgres-ha scale statefulset/backup-lab --replicas=1
kubectl -n postgres-ha rollout status statefulset/backup-lab --timeout=10m

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c "TABLE orders;"
```

Expected: Rows 1–4 appear after WAL replay.

2.6 Point-in-Time Recovery (PITR)

```
# Take a new full backup
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
# → Note the latest full backup label ending with F

# Insert a good row and record the timestamp
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c \
  "INSERT INTO orders VALUES (5,'good row');"

TARGET_TIME=$(kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -Atqc \
  "SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC',
  'YYYY-MM-DD HH24:MI:SS.US') || '+00';")

echo "PITR target: $TARGET_TIME"

# Simulate accidental DELETE
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab -c "DELETE FROM orders;"

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -c "SELECT pg_switch_wal();"
```

Scale down, create recovery pod, restore to target time:

```
kubectl -n postgres-ha scale statefulset/backup-lab --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 --timeout=180s
kubectl apply -f backup-lab-recovery.yaml
kubectl -n postgres-ha wait --for=condition=Ready pod/backup-lab-recovery --timeout=10m

kubectl -n postgres-ha exec backup-lab-recovery -- \
  bash -ec "mv /var/lib/postgresql/data/pgdata \
  /var/lib/postgresql/data/pgdata.before-pitr && \
  install -d -o postgres -g postgres /var/lib/postgresql/data/pgdata"

# Replace FULL_LABEL_HERE with the actual full backup label from info
kubectl -n postgres-ha exec backup-lab-recovery -- \
  gosu postgres pgbackrest --stanza=lab \
  --set=FULL_LABEL_HERE --type=time \
  "--target=$TARGET_TIME" --target-action=promote restore

kubectl -n postgres-ha delete pod backup-lab-recovery --wait=true
kubectl -n postgres-ha scale statefulset/backup-lab --replicas=1
kubectl -n postgres-ha rollout status statefulset/backup-lab --timeout=10m

kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d drlab \
  -c "SELECT * FROM orders ORDER BY id;"
```

Expected: Rows 1–5 present; the DELETE is not replayed. `pg_is_in_recovery()` should return false. A new timeline is created.

2.7 Compression, Retention & Cleanup

The ConfigMap already enables gzip compression (`compress-type=gz`) and retains two full backups (`repo1-retention-full=2`). Encryption (`repo1-cipher-type=aes-256-cbc`) must be configured **before** creating a fresh repository; do not change it on an existing repo.

Cleanup when the exercise is finished:

```
kubectl -n postgres-ha scale statefulset/backup-lab --replicas=0
kubectl -n postgres-ha wait --for=delete pod/backup-lab-0 --timeout=180s
kubectl -n postgres-ha delete statefulset backup-lab
kubectl -n postgres-ha delete configmap backup-lab-config
kubectl -n postgres-ha delete secret backup-lab-auth

# Optional - only after confirming you no longer need the backups
# kubectl -n postgres-ha delete pvc data-backup-lab-0 backup-lab-repo

# Delete the test recovery cluster from Part 1
kubectl delete postgresql acid-recovery-test
```

Coverage Checklist

Topic	Part 1 (Live)	Part 2 (Lab)
Logical Backup	Yes	Yes
Physical Backup	Partial (pg_basebackup / operator)	Yes (pgBackRest)
WAL Archiving	Check status	Yes (full config)
PITR	Pattern only	Yes (hands-on)
pgBackRest	No	Yes
Repository	—	Yes
Compression	—	Yes (gzip)
Encryption	—	Discussed
Incremental Backup	—	Yes
Differential Backup	—	Yes
Retention Policies	—	Yes
Configure pgBackRest	—	Yes
Full / Incr / Diff	—	Yes
Restore Database	Yes (test cluster)	Yes
PITR Recovery	Pattern	Yes

References

- pgBackRest User Guide — <https://pgbackrest.org/user-guide.html>
- Zalando Postgres Operator — <https://postgres-operator.readthedocs.io/en/latest/administrator/>
- Kubernetes StatefulSets — <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>
- PostgreSQL 18 Continuous Archiving — <https://www.postgresql.org/docs/18/continuous-archiving.html>

End of Lab Manual — Execute Part 1 first on the live cluster, then Part 2 for full enterprise feature coverage.